

Computer Networks LAB

LAB 3 Echo_Server

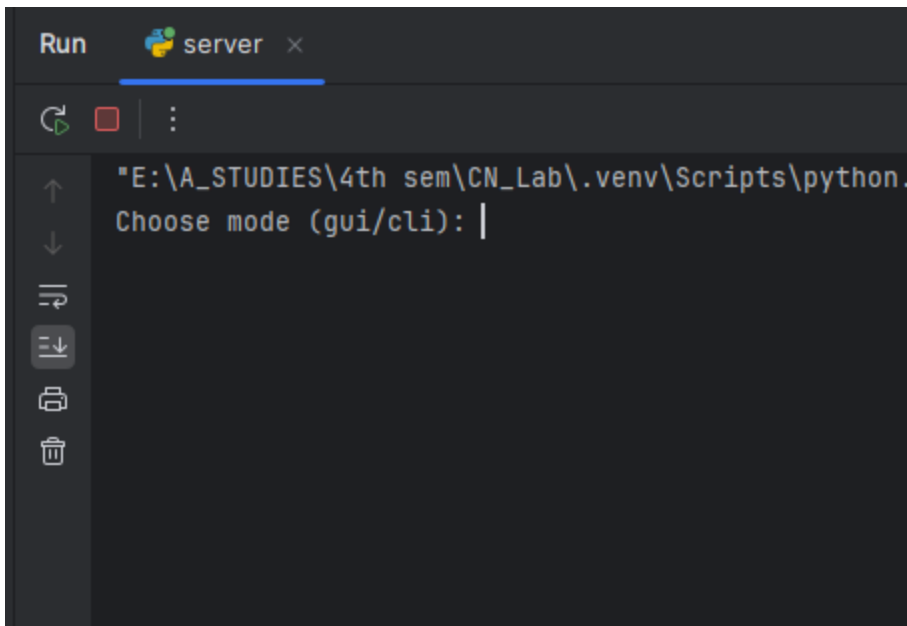
Aravinda Kannan KS (24BCE1290)

In Collaboration with : S Tharun Parykshyt (24BCE1119)

Menu Based Gui/Cli selection

This menu exists to keep the project flexible instead of locking it into a single interaction style. The same networking logic is reused everywhere; only the way the user interacts with it changes.

- GUI modes focus on usability and visual feedback
- CLI modes focus on speed, debugging, and simplicity
- Switching between modes does **not** affect socket behavior



GUI Based Server

The GUI-based server acts as a control panel, not the server itself.

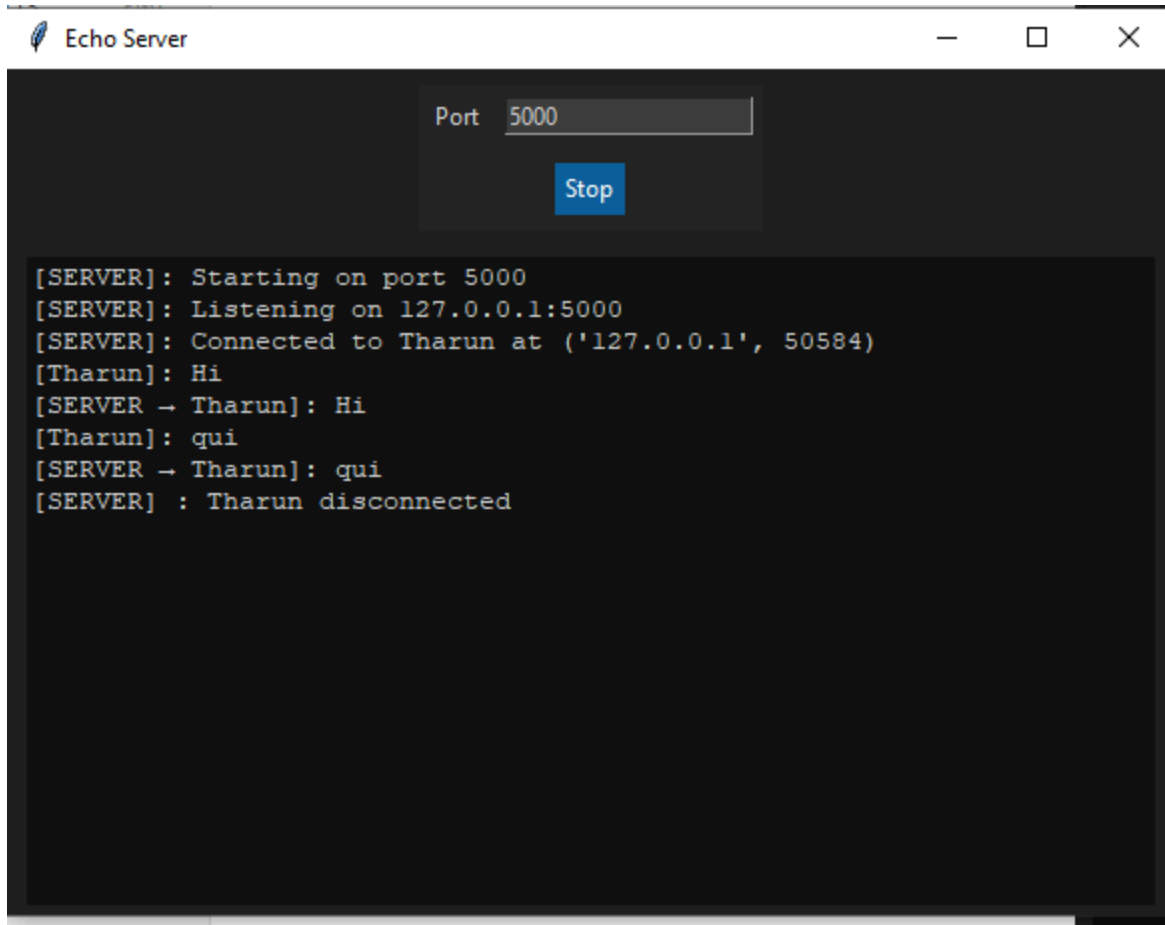
It allows:

- Starting and stopping the server safely
- Viewing server logs in real time
- Running the server without terminal commands

It intentionally avoids:

- Direct socket handling
- Blocking operations on the UI thread

All networking runs in the background to keep the interface responsive.



CLI Based Client

The CLI client is deliberately minimal.

Its purpose is to:

- Quickly test server connectivity
- Observe raw request/response behavior
- Debug protocol-level issues

This mode is especially useful during development when a full GUI is unnecessary.

```
Choose mode (gui/cli): cli
Enter username: Tharun
Enter server IP: 127.0.0.1
Enter server port: 5000
[server]: Connected to 127.0.0.1:5000
Enter message (exit to quit): Hi
[server]: Hi
Enter message (exit to quit): qui
[server]: qui
Enter message (exit to quit): exit

Process finished with exit code 0
```

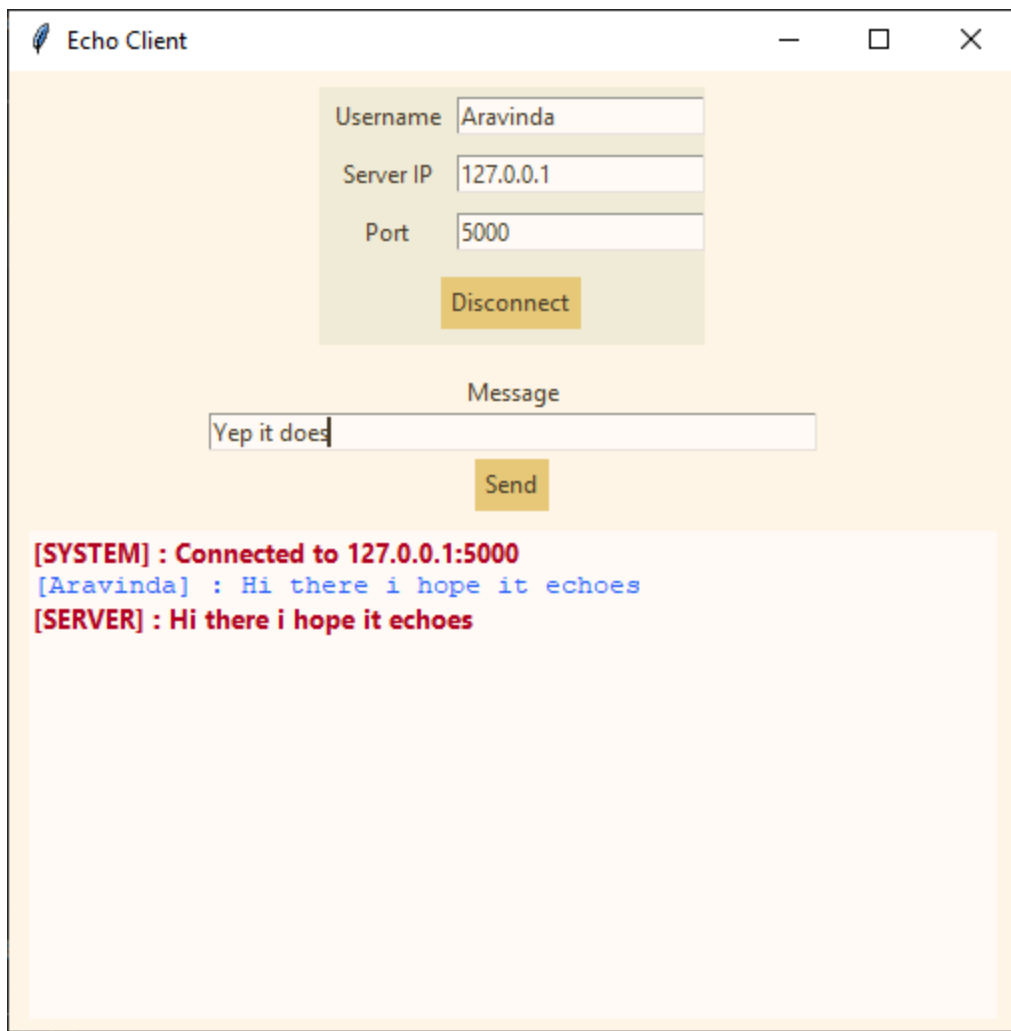
GUI Based Client

The GUI-based client focuses entirely on user interaction.

Its responsibilities include:

- Collecting connection details
- Displaying incoming and outgoing messages
- Clearly reflecting connection state

All socket operations are handled by backend logic, which prevents UI freezes and threading issues.



CLI Based Server

The CLI server provides the simplest way to run the backend.

Common use cases:

- Headless environments
- Rapid testing
- Running the server without GUI overhead

Functionally, it behaves the same as the GUI server.

```
"E:\A_STUDIES\4th sem\CN_Lab\.venv\Scripts\python.exe" "E:\
Choose mode (gui/cli): cli
Enter server port to listen on: 5000
[SERVER]: Listening on 127.0.0.1:5000
[SERVER]: Connected to Aravinda at ('127.0.0.1', 50627)
[Aravinda]: Hi there i hope it echoes
[SERVER → Aravinda]: Hi there i hope it echoes
|
```

Frontend Code Explanation

GUI for Client

```
# Import tkinter for GUI components
import tkinter as tk

# messagebox is used for popup error dialogs
from tkinter import messagebox

# threading allows background listening without freezing the GUI
import threading

# Type hints for better readability and static checking
from typing import Type, Optional

class ClientGUI:
    """
    Tkinter GUI for Echo Client.
    This class ONLY handles the UI.
    Networking / socket logic is delegated to the client_class.
    """

    def __init__(self, client_class: Type) -> None:
        # Store the client class (not instance yet)
        self.client_class: Type = client_class

        # Will hold the actual client instance after connecting
        self.client: Optional[object] = None

        # Tracks connection state
        self.connected: bool = False

        # ---- Color theme definitions ----
```

```

BG = "#FFF8E7"           # Main background
PANEL = "#F3EED9"        # Panel background
FG = "#4A3F2A"           # Text color
ENTRY_BG = "#FFDF6"      # Entry background
BTN = "#E6C97A"          # Button color
BTN_ACTIVE = "#D9B85C"   # Button active color

# Create main Tk window
self.root: tk.Tk = tk.Tk()
self.root.title("Echo Client")
self.root.configure(bg=BG)

# ---- Connection details frame ----
frame = tk.Frame(self.root, bg=PANEL)
frame.pack(pady=8)

# Username label + entry
tk.Label(frame, text="Username", bg=PANEL, fg=FG).grid(row=0, column=0,
padx=5, pady=4)
self.username_entry = tk.Entry(frame, bg=ENTRY_BG, fg=FG,
insertbackground=FG)
self.username_entry.grid(row=0, column=1)

# Server IP label + entry
tk.Label(frame, text="Server IP", bg=PANEL, fg=FG).grid(row=1, column=0,
padx=5, pady=4)
self.ip_entry = tk.Entry(frame, bg=ENTRY_BG, fg=FG, insertbackground=FG)
self.ip_entry.insert(0, "127.0.0.1") # Default to localhost
self.ip_entry.grid(row=1, column=1)

# Port label + entry
tk.Label(frame, text="Port", bg=PANEL, fg=FG).grid(row=2, column=0, padx=5,
pady=4)
self.port_entry = tk.Entry(frame, bg=ENTRY_BG, fg=FG, insertbackground=FG)
self.port_entry.insert(0, "5000") # Default port
self.port_entry.grid(row=2, column=1)

# Connect / Disconnect button
self.connect_btn = tk.Button(
    frame,
    text="Connect",
    command=self.toggle_connection,
    bg=BTN,
    fg=FG,
    activebackground=BTN_ACTIVE,
    relief="flat"

```

```

)
self.connect_btn.grid(row=3, column=0, columnspan=2, pady=8)

# ---- Messaging section ----
tk.Label(self.root, text="Message", bg=BG, fg=FG).pack(pady=(5, 0))

# Entry for typing messages (disabled until connected)
self.entry = tk.Entry(
    self.root,
    width=50,
    state="disabled",
    bg=ENTRY_BG,
    fg=FG,
    insertbackground=FG
)
self.entry.pack()

# Send button (disabled until connected)
self.send_btn = tk.Button(
    self.root,
    text="Send",
    state="disabled",
    command=self.send_msg,
    bg=BTN,
    fg=FG,
    activebackground=BTN_ACTIVE,
    relief="flat"
)
self.send_btn.pack(pady=4)

# Text widget to display chat output
self.output = tk.Text(
    self.root,
    height=15,
    width=60,
    bg=ENTRY_BG,
    fg=FG,
    insertbackground=FG,
    relief="flat"
)

# ---- Text styling tags ----
# Messages sent by the user
self.output.tag_configure("user", foreground="#1E5EFF")

# System / server messages

```

```

self.output.tag_configure(
    "system",
    foreground="#B00020",
    font=("Segoe UI", 10, "bold")
)

self.output.pack(padx=10, pady=6)

def write_log(self, msg: str, tag: str = "system") -> None:
    """
    Writes a message to the output box with a specific style tag.
    Auto-scrolls to the bottom.
    """
    self.output.insert("end", msg + "\n", tag)
    self.output.see("end")

# ----- LISTENER THREAD -----

def listen_loop(self) -> None:
    """
    Runs in a background thread.
    Continuously listens for messages from the server.
    """
    while self.connected and self.client:
        msg = self.client.receive()

        # None means connection closed or error
        if msg is None:
            break

        # Display server message
        self.write_log(f"[SERVER] : {msg}", "system")

    # Cleanup after disconnect
    self.on_disconnect()

def on_disconnect(self) -> None:
    """
    Resets UI state when connection is lost or closed.
    """
    if not self.connected:
        return

    self.connected = False
    self.entry.config(state="disabled")
    self.send_btn.config(state="disabled")

```



```

self.connect_btn.config(text="Connect")
self.write_log("[SYSTEM] : Disconnected", "system")

# ----- CONNECTION HANDLING -----

def toggle_connection(self) -> None:
    """
    Connects if disconnected.
    Disconnects if already connected.
    """
    if not self.connected:
        # Read user inputs
        username = self.username_entry.get().strip()
        ip = self.ip_entry.get().strip()

        # Validate port
        try:
            port = int(self.port_entry.get().strip())
        except ValueError:
            messagebox.showerror("Error", "Port must be a number")
            return

        # Username is mandatory
        if not username:
            messagebox.showerror("Error", "Username required")
            return

        # Create client instance
        self.client = self.client_class(username)

        # Attempt connection
        try:
            self.client.connect(ip, port)
        except Exception as e:
            messagebox.showerror("Error", f"Connection failed: {e}")
            return

        # Update UI state on success
        self.connected = True
        self.connect_btn.config(text="Disconnect")
        self.entry.config(state="normal")
        self.send_btn.config(state="normal")

        # Start listener thread
        threading.Thread(target=self.listen_loop, daemon=True).start()
        self.write_log(f"[SYSTEM] : Connected to {ip}:{port}", "system")

```

```

    else:
        # Disconnect logic
        if self.client:
            self.client.close()
        self.on_disconnect()

# ----- MESSAGE SENDING -----

def send_msg(self) -> None:
    """
    Sends the message typed by the user to the server.
    """
    if not self.connected or not self.client:
        return

    msg = self.entry.get().strip()
    if not msg:
        return

    # Clear input field
    self.entry.delete(0, "end")

    # Show message in UI immediately
    self.write_log(f"[{self.client.username}] : {msg}", "user")

    # Send message to server
    try:
        self.client.send(msg)
    except Exception:
        self.on_disconnect()

def start(self) -> None:
    """
    Starts the Tkinter event loop.
    """
    self.root.mainloop()

```

GUI for Server

```

import tkinter as tk
import threading
from typing import Optional
from server_logic import EchoServer

```

```

class ServerGUI:
    """
    Tkinter GUI for Echo Server.

    Responsibility boundaries:
    - This class handles ONLY the frontend (UI + user interaction).
    - It does NOT implement networking logic.
    - The actual server runs in a background thread via EchoServer.
    """

    def __init__(self) -> None:
        # Holds the EchoServer instance once started
        self.server: Optional[EchoServer] = None

        # Thread in which the server runs (so GUI stays responsive)
        self.server_thread: Optional[threading.Thread] = None

        # UI-level state flag (NOT server socket state)
        self.running: bool = False

        # ---- Color palette (pure UI concern) ----
        BG = "#1e1e1e"      # Main window background
        PANEL = "#252526"   # Control panel background
        FG = "#d4d4d4"      # Text color
        BTN = "#0e639c"     # Button color
        ENTRY_BG = "#3c3c3c" # Entry field background

        # Create root Tk window (must run on main thread)
        self.root: tk.Tk = tk.Tk()
        self.root.title("Echo Server")
        self.root.configure(bg=BG)

        # ---- Port selection panel ----
        frame = tk.Frame(self.root, bg=PANEL)
        frame.pack(pady=8)

        # Port label (purely informational UI element)
        tk.Label(
            frame,
            text="Port",
            bg=PANEL,
            fg=FG
        ).grid(row=0, column=0, padx=5, pady=5)

        # Entry where user specifies which port the server listens on

```

```

self.port_entry = tk.Entry(
    frame,
    bg=ENTRY_BG,
    fg=FG,
    insertbackground=FG # Cursor color
)
self.port_entry.insert(0, "5000") # Default value
self.port_entry.grid(row=0, column=1, padx=5, pady=5)

# Start / Stop button
# Single button toggles server state
self.start_btn = tk.Button(
    frame,
    text="Start",
    command=self.toggle_server, # UI event handler
    bg=BTN,
    fg="white",
    activebackground="#1177bb",
    activeforeground="white",
    relief="flat"
)
self.start_btn.grid(row=1, column=0, columnspan=2, pady=8)

# ---- Log window ----
# Displays server-side events (connections, messages, shutdowns)
self.log = tk.Text(
    self.root,
    height=20,
    width=70,
    bg="#111111",
    fg=FG,
    insertbackground=FG,
    relief="flat"
)
self.log.pack(padx=10, pady=5)

# ----- LOGGING -----

def write_log(self, msg: str) -> None:
    """
    Thread-safe log writer.

    Why this exists:
    - Tkinter is NOT thread-safe
    - Background threads (server thread) must NOT touch UI widgets
    - root.after(...) schedules execution on the main GUI thread

```

```

    """
    self.root.after(0, self._append_log, msg)

def _append_log(self, msg: str) -> None:
    """
    Actually modifies the Text widget.
    This method is guaranteed to run on the GUI thread.
    """
    self.log.insert("end", msg + "\n")
    self.log.see("end") # Auto-scroll to latest entry

# ----- SERVER CONTROL -----

def start_server(self, port: int) -> None:
    """
    Runs in a BACKGROUND THREAD.

    Purpose:
    - Creates and starts the EchoServer
    - Keeps blocking socket operations off the GUI thread
    """
    self.server = EchoServer(port=port)

    # log_callback allows backend to push messages to frontend
    # GUI remains decoupled from server internals
    self.server.start(log_callback=self.write_log)

def toggle_server(self) -> None:
    """
    UI event handler for Start / Stop button.

    Controls:
    - Server lifecycle
    - Button label
    - UI state
    """
    if not self.running:
        # Parse port number from UI
        try:
            port = int(self.port_entry.get().strip())
        except ValueError:
            # UI-level validation feedback
            self.write_log("[SERVER]: Invalid port")
            return

        # Start server in a separate thread

```

```

        # daemon=True ensures thread exits when GUI closes
        self.server_thread = threading.Thread(
            target=self.start_server,
            args=(port,),
            daemon=True
        )
        self.server_thread.start()

        # Update UI state
        self.running = True
        self.start_btn.config(text="Stop")
        self.write_log(f"[SERVER]: Starting on port {port}")

    else:
        # Stop server safely
        if self.server:
            self.server.stop()
            self.write_log("[SERVER]: Stopped")

        # Update UI state
        self.running = False
        self.start_btn.config(text="Start")

def start(self) -> None:
    """
    Starts the Tkinter event loop.

    This MUST be called from the main thread.
    """
    self.root.mainloop()

```

Socket Logic Explained

Client

```

import socket
from typing import Optional

class EchoClient:
    """
    EchoClient encapsulates ALL networking responsibilities.

    Design principle:
    - This class represents the CLIENT side of a TCP connection.
    """

```

- It knows NOTHING about GUI, threading, or presentation.
 - It only knows how to:
 1. Establish a TCP connection
 2. Send bytes
 3. Receive bytes
 4. Handle disconnection safely
- """

```
def __init__(self, username: str) -> None:
    """
    username:
        - Application-level identity
        - TCP itself does NOT know about usernames
        - This is protocol data defined by *us*, not by TCP

    sock:
        - Represents a TCP endpoint (file-descriptor-like object)
        - None means: no connection exists yet

    connected:
        - Logical state flag maintained by application
        - TCP sockets alone do NOT provide a clean "connected?" API
        - We track it ourselves for correctness
    """
    self.username: str = username
    self.sock: Optional[socket.socket] = None
    self.connected: bool = False

def connect(self, host: str, port: int) -> None:
    """
    Establishes a TCP connection to the server.

    Key networking concepts here:
    - AF_INET      → IPv4 addressing (x.x.x.x)
    - SOCK_STREAM  → TCP (reliable, ordered, byte-stream)
    """

    # Create a TCP socket
    # At this point:
    # - No packets sent yet
    # - No connection exists
    self.sock = socket.socket(socket.AF_INET, socket.SOCK_STREAM)

    # connect() triggers the TCP 3-way handshake:
    # SYN → SYN-ACK → ACK
    # This blocks until the handshake succeeds or fails
```

```

self.sock.connect((host, port))

# After TCP connection is established,
# we immediately send application-level metadata (username)
#
# IMPORTANT:
# - TCP does NOT define message meaning
# - We define a custom protocol:
#   "first message after connect = username"
self.sock.sendall(self.username.encode())

# sendall():
# - Guarantees all bytes are sent or raises an error
# - Unlike send(), it does NOT allow partial sends
self.connected = True

def send(self, msg: str) -> None:
    """
    Sends a message to the server.

    Networking model used:
    - Blocking TCP send
    - Fire-and-forget (no response expected here)

    IMPORTANT:
    - TCP is a byte stream, NOT message-based
    - We assume messages are small and fit into one recv()
    """

    # Application-level validation
    # TCP socket existing does NOT mean connection is healthy
    if not self.connected or not self.sock:
        raise RuntimeError("Not connected to server")

    try:
        # Convert string -> bytes (TCP only transmits bytes)
        self.sock.sendall(msg.encode())

    except OSError:
        # Any OSError here usually means:
        # - Connection reset
        # - Broken pipe
        # - Server crash
        #
        # TCP does NOT raise a clean "disconnected" event
        # Errors are how disconnection is detected

```



```

        self.connected = False
        raise

def receive(self) -> Optional[str]:
    """
    Receives ONE chunk of data from the server.

    TCP receive properties:
    - recv() BLOCKS until data arrives or connection closes
    - recv(1024) does NOT guarantee message boundaries
    - It returns:
        * bytes > 0 → data received
        * b'' → peer closed connection cleanly
    """

    if not self.connected or not self.sock:
        return None

    try:
        # recv() reads from the TCP receive buffer
        # 1024 is a buffer size, NOT a message size
        data: bytes = self.sock.recv(1024)

        # If recv returns empty bytes:
        # - TCP FIN received
        # - Server closed connection gracefully
        if not data:
            self.connected = False
            return None

        # Decode bytes → string
        msg = data.decode()

        # Application-level control message
        # TCP does NOT support control signals like this
        # We define "__SERVER_SHUTDOWN__" as protocol semantics
        if msg == "__SERVER_SHUTDOWN__":
            self.connected = False
            return None

        return msg

    except OSError:
        # Any exception here means:
        # - Network error
        # - Connection aborted

```

```

        # - Socket invalid
        #
        # TCP failure detection is implicit via errors
        self.connected = False
        return None

def close(self) -> None:
    """
    Cleanly shuts down the TCP connection.

    Proper TCP teardown:
    - shutdown() → sends FIN in both directions
    - close()    → releases OS resources
    """

    self.connected = False

    if self.sock:
        try:
            # SHUT_RDWR:
            # - Disable both sending and receiving
            # - Notifies peer of intent to close
            self.sock.shutdown(socket.SHUT_RDWR)
        except OSError:
            # If already closed or broken, ignore
            pass

    # close() releases the file descriptor
    self.sock.close()
    self.sock = None

```

Server

```

import socket
import threading
from typing import Callable, List, Tuple, Optional

class EchoServer:
    """
    EchoServer implements a classic TCP concurrent server.

    Core networking model:
    - TCP (SOCK_STREAM)
    - One listening socket

```

- One dedicated thread per connected client
 - Blocking I/O with timeouts for graceful shutdown
- ```
"""
```

```
def __init__(self, host: str = "127.0.0.1", port: int = 5000) -> None:
 # IP address to bind the server socket to
 # "127.0.0.1" means loopback (local machine only)
 self.host: str = host

 # TCP port number the server listens on
 self.port: int = port

 # The listening socket (accepts new connections)
 self.server: Optional[socket.socket] = None

 # List of active clients
 # Each entry = (username, client_socket)
 #
 # NOTE:
 # TCP sockets do NOT store usernames
 # This mapping is purely application-level state
 self.clients: List[Tuple[str, socket.socket]] = []

 # Server lifecycle flag
 # Used to coordinate shutdown across threads
 self.running: bool = False

def start(self, log_callback: Callable[[str], None] = print) -> None:
 """
 Starts the TCP server and begins accepting connections.

 This method typically runs in a background thread
 so the GUI/main thread remains responsive.
 """

 # Create a TCP socket:
 # - AF_INET → IPv4
 # - SOCK_STREAM → TCP (reliable, ordered, connection-oriented)
 self.server = socket.socket(socket.AF_INET, socket.SOCK_STREAM)

 # Bind the socket to (host, port)
 # This reserves the port at the OS level
 self.server.bind((self.host, self.port))

 # Put socket into listening mode
 # Argument (5) = backlog
```

```

→ max queued incoming connections before accept()
self.server.listen(5)

Set timeout on accept()
WHY:
- accept() is blocking by default
- timeout allows periodic checks of self.running
- without this, server could never shut down cleanly
self.server.settimeout(1.0)

self.running = True
log_callback(f"[SERVER]: Listening on {self.host}:{self.port}")

Main accept loop
while self.running:
 try:
 # accept() blocks until:
 # - a client connects
 # - timeout expires
 # - socket is closed
 conn, addr = self.server.accept()

 except socket.timeout:
 # No incoming connection during timeout
 # Loop continues so we can re-check self.running
 continue

 except OSError:
 # Listening socket closed → server shutting down
 break

----- APPLICATION-LEVEL HANDSHAKE -----
try:
 # First recv() after accept()
 # We define this as "username"
 #
 # IMPORTANT:
 # TCP does NOT define message meaning
 # This protocol rule is OUR design
 username = conn.recv(1024).decode()

 # Empty recv means client closed immediately
 if not username:
 conn.close()
 continue

```

```

except Exception:
 # Any error during handshake → drop connection
 conn.close()
 continue

Store client socket + identity
self.clients.append((username, conn))
log_callback(f"[SERVER]: Connected to {username} at {addr}")

Spawn a new thread for this client
#
CONCURRENCY MODEL:
- One thread per client
- Each thread blocks on recv()
- Simple, readable, but not scalable to thousands
threading.Thread(
 target=self.handle_client,
 args=(username, conn, log_callback),
 daemon=True # thread exits automatically when main program exits
).start()

def handle_client(
 self,
 username: str,
 conn: socket.socket,
 log_callback: Callable[[str], None]
) -> None:
 """
 Handles all communication with ONE client.

 This function runs in its OWN thread.
 """

 # Set recv timeout so thread can:
 # - periodically check self.running
 # - exit cleanly during server shutdown
 conn.settimeout(1.0)

 while self.running:
 try:
 # recv() reads from TCP receive buffer
 # 1024 is a buffer size, NOT a message size
 data = conn.recv(1024)

 # Empty bytes means:
 # - TCP FIN received

```

```

- Client closed connection gracefully
if not data:
 break

Decode byte stream → application string
msg = data.decode()

----- NORMAL CHAT MESSAGE -----
log_callback(f"[{username}]: {msg}")

Echo message back to same client
#
sendall():
- Ensures entire buffer is sent
- Required for TCP correctness
conn.sendall(msg.encode())

log_callback(f"[SERVER → {username}]: {msg}")

except socket.timeout:
 # No data received during timeout window
 # Allows loop to continue and re-check self.running
 continue

except (ConnectionResetError, OSError):
 # ConnectionResetError:
 # - Client crashed or force-closed socket
 #
 # OSError:
 # - Network failure
 # - Socket invalid
 break

----- CLEAN DISCONNECT -----
IMPORTANT:
Client disconnect is NOT a message
It is detected via recv() returning empty bytes or error
log_callback(f"[SERVER] : {username} disconnected")

Close client socket
try:
 conn.close()
except Exception:
 pass

Remove client from active list

```

```

Required to avoid sending to dead sockets
self.clients = [(u, c) for u, c in self.clients if c != conn]

def stop(self) -> None:
 """
 Stops the server gracefully.

 Shutdown strategy:
 1. Stop accept loop
 2. Close listening socket
 3. Notify all clients
 4. Close all client sockets
 """

 # Signal all threads to stop
 self.running = False

 # --- Close listening socket ---
 # This unblocks accept()
 if self.server:
 try:
 self.server.close()
 except Exception:
 pass
 self.server = None

 # --- Notify and close all client connections ---
 for username, conn in self.clients:
 try:
 # Application-level control message
 # TCP has NO built-in shutdown notification
 conn.sendall("__SERVER_SHUTDOWN__".encode())
 except Exception:
 pass

 try:
 # shutdown() sends FIN in both directions
 conn.shutdown(socket.SHUT_RDWR)
 except Exception:
 pass

 try:
 conn.close()
 except Exception:
 pass

```

```
--- Thread cleanup (optional / defensive) ---
If client threads were tracked, wait for them
for t in getattr(self, "client_threads", []):
 t.join()

Clear internal state
self.clients.clear()
if hasattr(self, "client_threads"):
 self.client_threads.clear()
```